OBD = Open Barn Door? Security Vulnerabilities and Protections for Vehicular On-Board Diagnosis (OBD)

Stephanie Bayer, ESCRYPT GmbH
Leopoldstr. 244, 80807 München
stephanie.bayer@escrypt.com

Ramona Jung, ESCRYPT GmbH
L. M. Allee 4, 44801 Bochum
ramona.jung@escrypt.com

Marko Wolf, ESCRYPT GmbH
Leopoldstr. 244, 80807 München
marko.wolf@escrypt.com

Abstract

The automotive On-Board Diagnostic (OBD) interface enables a standardized, efficient way to access information of the in-vehicle electronic system. However, the standardization of this interface also facilitates unauthorized access to critical services. Implemented simple OBD security mechanisms can be easily circumvented and publicly available tools allow even untrained persons the execution of virtually all diagnostic services, for example to manipulate the mileage or to activate component features without authorization. This paper provides an overview of OBD security vulnerabilities and proposes some effective and efficient security solutions that not only protect the system against unauthorized access by the integration of state-of-the-art cryptography, but also consider in-vehicle application constraints.

1. Introduction and Motivation

Modern vehicles have become complex information technology (IT) systems consisting of multiple interconnected Electronic Control Units (ECUs) which are responsible for safe and correct functionality of the car. In order to provide comprehensive, easy-to-use self-diagnostic and reporting functionality for in-vehicle ECUs, a standardized On-Board Diagnostic (OBD) interface was developed in the 1990s and today OBD is deployed worldwide and legally mandatory in the US and Europe. On the one hand, OBD enables digital access to public data, for instance to emission control, error codes, and on the other hand, OBD enables access also to "hidden" manufacturer-specific ECU settings, for instance to theft protection or engine control.

Unfortunately, the widespread use of the OBD interface did not only ease maintenance but eases also unauthorized vehicle manipulations. Today, many tools are freely available on the market, which enable even beginners to tamper with the car. For example as shown in **Fehler! Verweisquelle konnte nicht gefunden werden.**, the mileage of a vehicle can be changed in a few seconds without leaving any traces by using a diagnostic tester [1]. Koscher et al. [2] describe how one can change in-vehicle parameters or circumvent the electronic locking system to affect even the driving behavior of the car. Even though the OBD standard foresees some basic mechanisms that enforce access control using four different security access levels, their practical realization is rather weak. Based on static, proprietary, or simply incorrect implemented algorithms, they often can be circumvented with cheap tools and only little publicly available knowledge [3].

The attractiveness for unauthorized access is even more increased due to the comprehensive and powerful functionality provided by "secret", which means non-standard manufacturer individual diagnostic services. By knowing corresponding "secret" OBD parameter identifiers, it is not only possible to obtain

detailed information from sensors and actuators not foreseen for disclosure, but also to modify critical ECU behavior, or even to reprogram the firmware or settings of some ECUs. Mileage alteration or unauthorized chip tuning are well-known practices which already exploit this vulnerability. Recent publications demonstrate some more advanced exploits by inserting manipulated parameters in order to "overtake" various safety-critical functionalities (e.g., steering, brakes) of a vehicle via OBD [3]. Other potential attackers might read out competitor's intellectual property (e.g., ECU firmware) or privacy invasive information (e.g., recent vehicle locations, eCall data set). Hence, there seems apparently a need for actions to protect the OBD interface against critical data security and privacy infringements.

First, we will provide some background on the OBD standard and the commonly used UDS protocol. Then we will give an overview of some security vulnerabilities of the OBD interface. After that, we will present efficient and effective protection mechanisms that will re-establish the security of a vehicle such as a mandatory access control mechanism that enforces fine-grained access authorizations or mutual authentication of vehicle and tester based on strong cryptographic primitives.

Figure 1: With OBD and weak UDS protection, manipulation of total vehicle mileage became very easy, cheap and virtually untraceable (Pictures: ADAC/Simon Koy).

2. Background Information on OBD and UDS

This section gives some introductory background details on OBD and UDS. If you are already familiar with these two automotive technologies, you can skip this section.

Automotive industry is using many different bus systems and diagnostic standards, among them the European On-Board-Diagnostic (EOBD) standard. Since 2004, EOBD is mandatory in Europe for all vehicles. This standard defines the diagnostic connector (OBD) (Figure 2), manufactory-independent Diagnostic Trouble Codes (DTC), and the coverage of the on-board diagnostics. Diagnostic communication is used to connect the in-vehicular diagnostic service with external diagnostic tools, so-called "*tester*" devices.

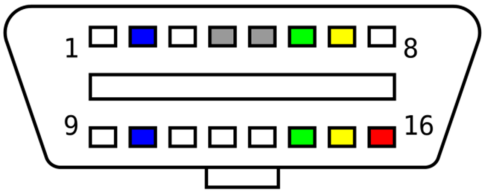

Figure 2: OBD connector shape and pin out with red = battery (+12Vcc), dark-gray = ground, blue = SAE J1850 (PWM/VPW), green = ISO 15765-4 (CAN), yellow = ISO 9141-2/14230-4 and white = manufacturer-specific function (Picture: Xoneca, CC0).

The tester can either query the current state of the vehicle or read out the OBD log file. In this file all noticeable behavior of the vehicle is reported and logged. In general, this refers to a malfunction of an

ECU caused by an application or service that is not being executed in the intended manner. Therefore, log files provide information about all in-vehicle incidents and can be used, for example, as evidence to an investigation in case of an accident.

One of the most used communication protocols over OBD is the Unified Diagnostic Services (UDS) protocol, which is implemented by many automotive manufacturers. The purpose of UDS is to get information from or set values in various ECUs inside a vehicle.

UDS is specified in ISO 14229-1 [4] and describes different messages and data sequences that allow, for instance, to establish different level of access, to read out error memory, or to update software on an ECU. For all this, UDS is the mutual foundation and allows creating a set of different diagnostic services, which can be used manufacturer-independently. However, the UDS standard only describes the functional interfaces and communication flows, but does not specify how the diagnostic services should be implemented in detail. The realization of diagnostic services itself is left to the automotive manufacturers. The UDS protocol was originally designed to be used over the common in-vehicle Controller Area Network (CAN) bus. The CAN bus was developed in 1983 for efficient and fast automotive communication and is standardized in ISO 11898 [5]. It broadcasts messages to all available receivers and restricts the message payload. However, today UDS is also used with other automotive communication protocols such as FlexRay [6]. Although all car manufactures use UDS via the physical OBD interface and a wired connection from tester to the vehicle, many car manufacturers today also support remote diagnostic services via WiFi or mobile communication interfaces [7]. This allows for instance to inspect a broken down car while the mechanic is driving to the stranded vehicle and thus speed up the repairing process rapidly.

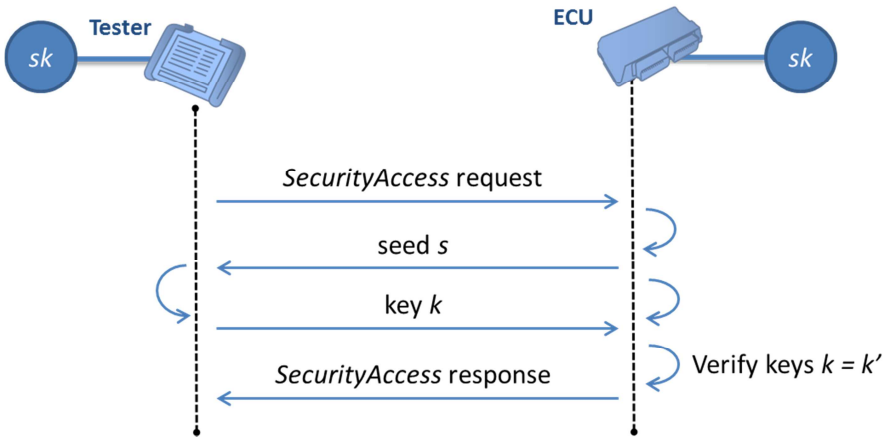

Figure 3: Seed-Key Protocol used in UDS *SecurityAccess* Service.

For optional access control and basic authentication to enhanced diagnostic services, a "SecurityAccess" service is defined which is based on a "Seed-and-Key" protocol. The general communication flow according to this protocol is depicted in **Fehler! Verweisquelle konnte nicht gefunden werden.**. An authentication of a diagnostic tester to an ECU is performed as follows. After receiving a *SecurityAccess* service request, the ECU generates a value called the "seed", and sends it to the tester. Using this *seed*, the tester then computes a corresponding value called the "key" and sends it back to the ECU. On the ECU's side, a reference value for the *key* is computed. If and only if the received *key* and the reference value are

equal, the tester is allowed to access enhanced diagnostic services on the ECU, for instance, for firmware updates.

3. Open Barn Door – OBD Security Threats and Vulnerabilities

OBD was developed to have a manufacturer independent diagnosis interface which enables easy access to diagnostic data and convenient reporting functionality of the in-vehicular ECUs. However, security was not specified at that time and therefore OBD offers virtual no protection against misuse. Hence, the diagnostic interface is not only useful for authorized garages, but also enables unauthorized persons to access in-vehicular ECUs and diagnostic services. This can lead to various security attacks, as already demonstrated by various researchers (e.g., [2] [3] [8]).

Due to commonly available tools and information, automotive manufacturers are faced today with an increasing amount of ECU manipulations that might affect vehicle safety, legal applications (e.g., digital tachograph, exhaust gas treatment), or undermine aftermarket business models. Thus, not only ambitioned vehicle owner can misuse diagnostic services for illegal feature activation, ECU parameter manipulations, or mileage reset. Also criminal organizations aim to retrieve information about the intellectual property of OEMs and suppliers for creating counterfeit components or about driver sensitive data as, such as driving behavior.

This chapter presents some exemplary security threats and vulnerabilities of OBD and UDS that might cause critical damages to OEMs, suppliers, drivers, owners, and environment and society.

3.1. Direct Access to Unprotected In-Vehicle CAN Bus

The OBD interface connects the outside world with the unprotected in-vehicular communication network, which was invented as closed internal system without allowing external access. The physical interface is usually placed at an easy accessible location in the vehicle, for example beneath the steering wheel. In addition, the standardization of the OBD interface offers the possibility to plug in common available connector devices, which can be used not only at vehicles of a dedicated manufacturer but for various models.

The attachment of an OBD connector enables direct access to the in-vehicle CAN bus system. CAN is a multi-cast protocol, which means that a message from any CAN node is always sent to all other nodes in the CAN network. This protocol property avoid costly message addressing and encryption overhead, but makes it also easy for an attacker to wiretap all messages, which are broadcasted inside the CAN network, by just plugging into the OBD interface. These wiretapped messages, however, can contain sensitive information such as the current vehicle speed. An attacker can also use the OBD connection to suppress or manipulate safety-critical messages [3]. Furthermore, a potential attacker can easily inject any own messages over the interface into the CAN network. The messages will be broadcast to all available receivers inside the network without any chance to verify the authorization of the sender. This allows an attacker to send arbitrary commands to the ECUs inside the network [2].

3.2. Weak *SecurityAccess* Service Implementation

Some diagnostic functions defined in the UDS standard (e.g., firmware update) are required to be only available for authorized users. To verify user authorization UDS provides a so-called *Security Access* protection mechanism based on the "Seed-and-Key" protocol as described in Section 0.

However, the UDS standard does not specify in detail how the *SecurityAccess* should be designed. They leave it open if cryptographic algorithms are used and give no recommendations which cryptographic algorithms should be used. However, the UDS standard recommends to calculate a new seed value for each access request since a reuse of the same seed, allows an attacker to circumvent the access functionality by replaying recorded data without knowing the underlying algorithm very easily. Unfortunately, this recommendation is not implemented every time, and either constant value seeds are used. Or the seed is picked from a small set of values, in this case the attacker also does not need any information of the underlying algorithm but has to eavesdrop a few valid access protocols before the attacker is able to reuse the gained knowledge. Another problem is that often the seeds and the key values are only 16 bit long [2] and therefore the access can be broken using Brute Force. An attacker with access to the vehicle can test all possible combinations of the seed and the key, the authors of [2] describe that they were able to carry out the attack in around 7.5 days.

Even if the seed is picked at random from a big set to avoid all former mentioned problems, the calculation of the key is often based on weak algorithms, which apply a linear function on the seed and therefore the key is predictable for an attacker who is in possession of the key generation algorithm. After the attacker has seen a few valid seed and key pairs, he is able to solve a linear system of equation to extract the secret value used to calculate the key. The discussion reveals that a basic implementation of the *SecurityAccess* service cannot provide a high security level. Hence, it is important to base the *SecurityAccess* on strong cryptographic algorithms.

3.3. Hijacking Authorized UDS Sessions

As described in Section 0, some critical UDS diagnostic services (e.g., firmware update) can be restricted to authorized users only by applying the *SecurityAccess* protection mechanism. Even under the assumption of a perfectly secure *SecurityAccess* mechanism (cf. Section 3.2), a successfully authorized diagnostic session could be easily hijacked by an unauthorized third party having CAN access.

This security vulnerability arises from the CAN protocol. As discussed before (cf. Section 3.1), CAN always broadcasts all messages to every ECUs inside the CAN network and it is not possible for a CAN receiver to verify the sender of a CAN message. Hence, once an authorized user has successfully created a UDS session to a certain ECU over CAN using *SecurityAccess*, anyone who has access to this CAN network, can overtake this session and send any commands he wants. The *Programming Session* of the UDS protocol permits access to the reprogramming functionality of an ECU. This could be misused by an attacker to manipulate the software of an ECU or change the software of the ECU completely. A re-programming device is inserted inside the in-vehicular network by the attacker and the device monitors the status of the target ECU. If the Programming Session is activated, the device hijacks the active session and sends its malicious code to the ECU, which then executes the malicious code after restart.

3.4. Globally Unique Secrets Keys

The “Seed-and-Key” algorithm applies a secret key value to calculate the response key from the seed. Only tester in possession of the correct secret value of the ECU can answer the seed correctly and gain access to the diagnostic service. The problem is, that the same secret key value is often used for a whole production line of a vehicle, i.e. the same ECU in all cars are using the same secret key value and sometimes this secret key value is shared by many ECUs. If an attacker can get hold of this globally used secret key value, he is able to access the diagnostic service of the whole production line. There are various ways to get hold of the secret key value, for instance from the garage or by extracting the secret key value from a tester.

3.5. Secret for key computation stored in unprotected memory

One of the most important rules of modern cryptology is that the security of cryptographic schemes should be based on the secrecy of the secret key material and not on the secrecy of the algorithm. However, often the “Seed-and-Key” algorithms used by the manufacturers are proprietary and are considered as confidential. Consequently, the manufactures assume that their diagnostic *SecurityAccess* implementation is secure as long as the algorithm for secret key computation remains secret. However, as discussed in Section 3.2, in many cases an attacker is able to circumvent the *SecurityAccess* protocol due to the weak design of the access protocol.

Another problem is that the secret key material is often stored in unprotected memory within the tester or within the ECU. An attacker who has access to a tester device or ECU can read out the secret material from this device, for instance, by gaining access to a tester over a debug interface. The possession of the secret key material enables now the attacker to execute the authentication in the *SecurityAccess* protocol successfully without using the tester.

3.6. Single User Account

The UDS standard allows different diagnostic session where each session supports different diagnostic services and each diagnostic service should be only available in the corresponding session.

In particular, the different sessions can be used to allocate different access rights for different authorities and therefore control the services an entity is allowed to access. However, often there are no different access rights allocated and users, garage person, police officers, or legal authorities can access the same service. Hence, an owner of a car might manipulate the mileage or re-program an ECU or execute another processes that should be only executable for an appointed set of technicians. Furthermore, a technician might read privacy related data, which should only be disclosed to legal authorities.

4. Closing the Barn Door – OBD Security Solutions

The previous chapter discussed various potential weaknesses of the OBD system. This chapter will counter the weaknesses and propose some protection mechanisms that will ensure the security of a vehicle A mandatory access control mechanism is introduced that enforces fine-grained access authorizations as well as mutual authentication of vehicle and tester based on strong cryptographic primitives.

The foundation of the protection lies in cryptographic strong authentication (cf. Section 4.1) which controls access to an ECU in fine-grained way and ensures that only approved roles have access to the diagnostic services. The second level of the defense is built up on secure communication. An authentication and encryption of the communication prohibits hijacking of established diagnostic sessions. Attacks can be prevented further by using firewalls and authenticated log files. But the security of the whole system is only as good as the storage of the keys. Therefore, protected key storage is needed together with a smart key management to ensure the security of the diagnostic access protocol.

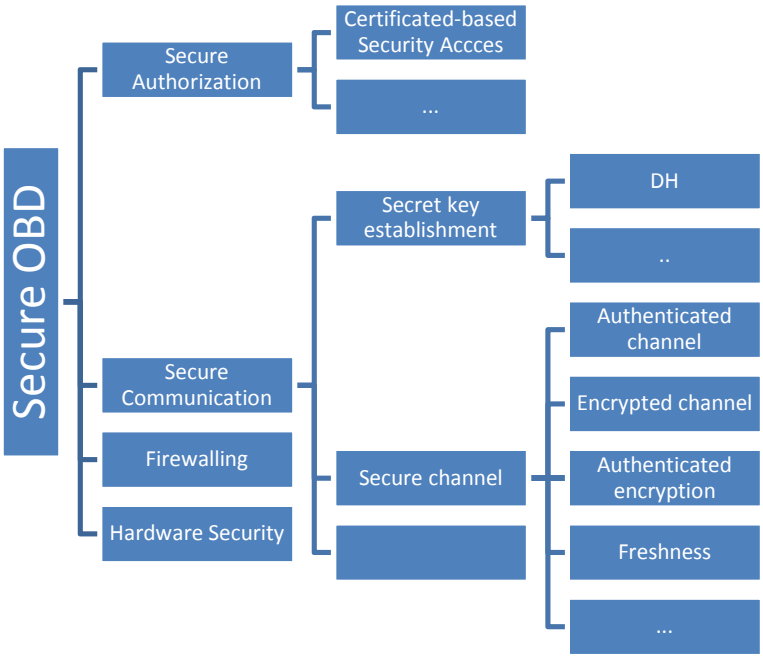

Figure 4: Overview of OBD security measures

4.1. Secure User Authentication & Access Control

One of the biggest problems of the diagnostic interface is the insufficient authentication of the tester device by the ECU such that virtually everybody can pretend to be an authorized tester. The UDS protocol by itself provides no possibility for strong authentication, but rather permits the usage of diagnosis functionality after successful execution of the *SecurityAccess*. However, the simple ”Seed-and-Key” protocol defined for secure access in UDS is far behind the security required today (cf. Section 3.2).

By applying a state-of-the-art bidirectional authentication protocol, it is possible to verify the testers authorization before accessing the ECU for a diagnostic session. A strong tester authentication can be done in various ways as shown for instance in [9]. This paper focus on one method, which enables a trusted third party to verify the authentication and to grant access rights.

For this solution, a Public Key Infrastructure (PKI) with a trusted Certificate Authority (CA) is needed. PKIs manage, issue, and revoke digital certificates, which consist mainly of a User-ID and a public key. The task of the trusted CA is to issue valid certificates by signing the data using its private key and to verify and revoke certificates. Using the properties of certificates a simple “Seed-and-Key” protocol can be constructed, which enables an entity A to authenticate itself against another entity B.

In more detail, B knows the public key of the CA, trusts the validity of it, and trusts the CA to only sign legitimate certificates. A owns a signed certificate with the corresponding public/private key pair. Having received an access request, B checks first the correctness of the public key by verifying the certificate using the CA's public key. After A has received the seed, it will calculate a signature for it using its private key. A sends this signature together with its public key certificate to B and B verifies the correctness of the signature for the challenge using A's public key.

This simple "Seed-and-Key" protocol can be used within the more complex protocol between an ECU and a tester as shown in **Fehler! Verweisquelle konnte nicht gefunden werden.** for more details. Each ECU is in possession of its own signed cryptographic certificate, which contains the ECU-ID and the ECU's public key. Using a "Seed-and-Key" protocol, as described before, the ECU can authenticate itself against the tester. Then the tester authenticates itself against the manufacturer backend using its own certificate. After successful authentication, the tester forwards the ECU's certificate to the backend. The backend can now decide whether the tester is allowed to gain the permission to access the ECU. This permission is granted in form of a new certificate that has been issued by the backend and that contains the tester-ID, the tester's public key, and a list of permitted services. Finally, the tester can use this certificate to authenticate itself against the ECU using a "Seed-and-Key" protocol and to get access to the permitted services.

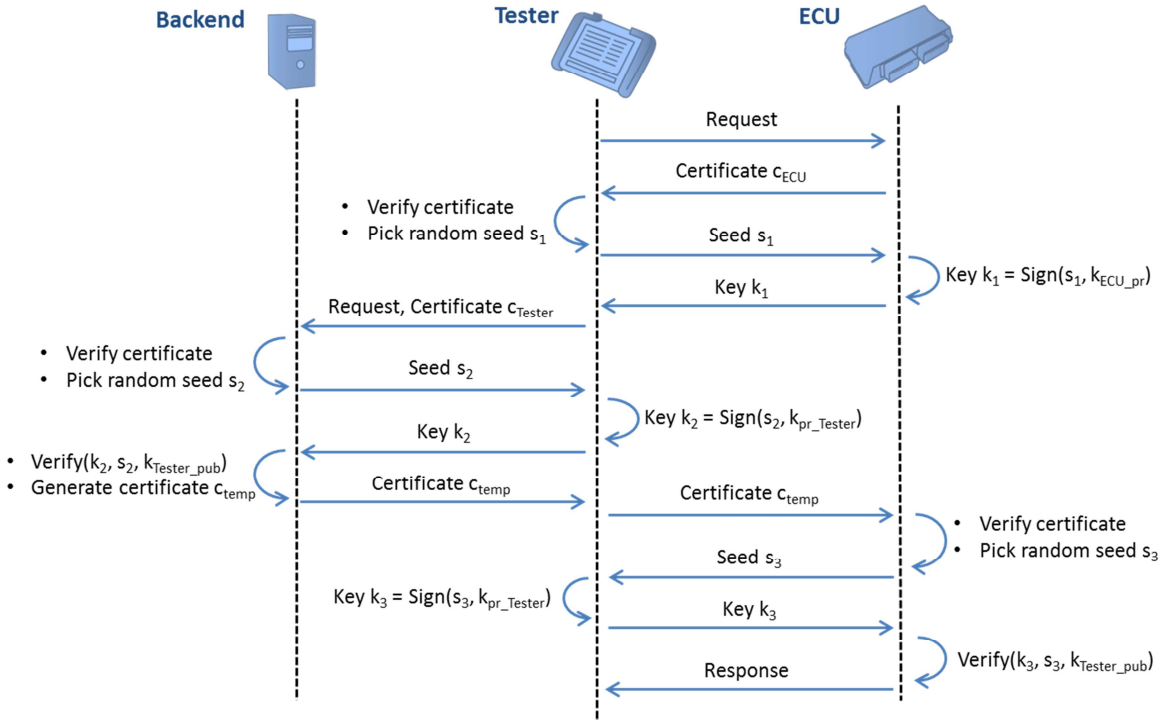

Figure 5: SecurityAccess protocol with manufacturer backend

One further advantage of this solution is the provision of fine-grained access control. Since UDS allows different access levels (cf. Section 3.6), this can be used to assign each tester a certain access level. The backend decides if the tester has access to the requested service. This means, in the case that the tester has the required access level or higher permission the backend grants access, otherwise it denies the access.

Communication after a successful authentication is, however, still unsecured. An attacker can hijack the session after a successful authentication, and could perform potentially harmful actions (cf. Section 3.3). The attacker simply needs to wait for a successful, legitimate authentication and can then send his own commands. This attack would, for example, be applicable in the case of chip tuning, where the owner of the car plants an attack device on the CAN bus before a regular garage check-up; another example is that a virus on an infected ECU uses that check-up for spreading to other ECUs. To prevent this kind of attacks the communication could be secured further by applying authenticity and confidentiality measures (cf. Section 4.2).

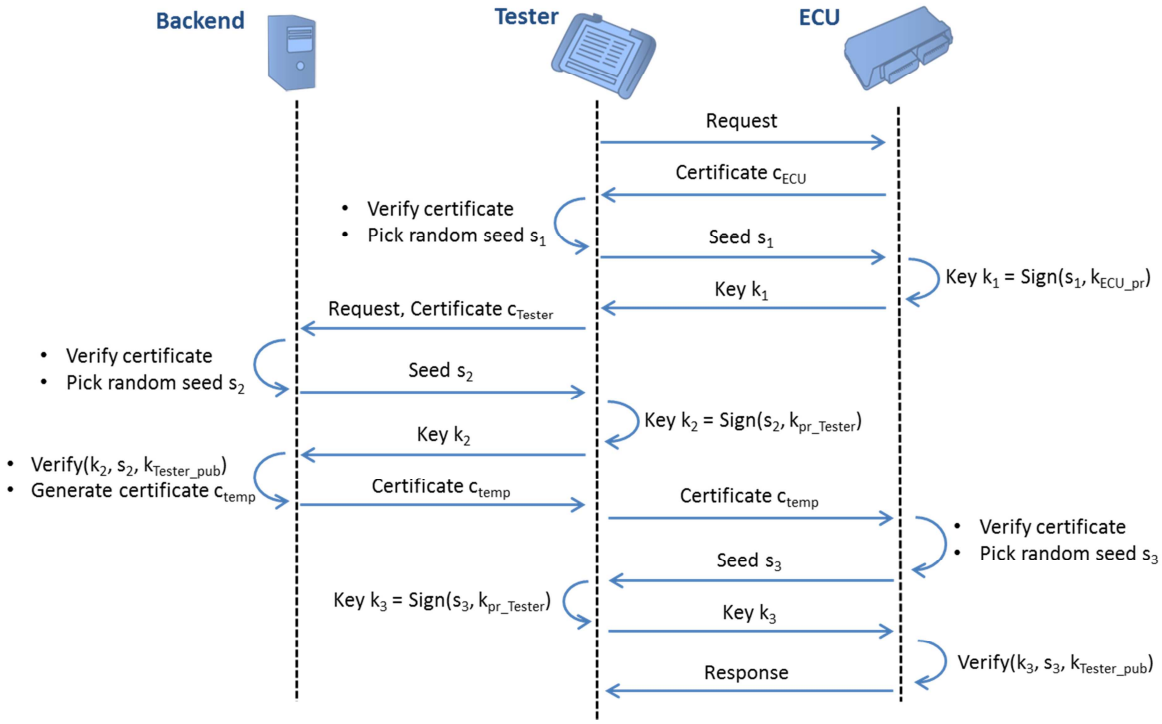

Figure 6: SecurityAcess protocol with manufacturer backend

4.2. Secure Communication Channel

To prevent a hijack of a successful authenticated session, it is advisable to secure also the communication channel itself. While UDS does not specify any means of secured communication channels, there are possibilities for the vehicle manufacturer to specify their own services and sessions with their own message flow – since nothing contradicts the use of a better-secured communication protocol on top. We can still use the *SecurityAccess* service for the authentication and key agreement necessary to establish such a secured channel.

Such a channel can for instance ensure the authenticity and integrity of the communication against illegitimate modifications by applying a symmetric MAC (Message Authentication Code). In addition, the channel can also ensure the confidentiality of the messages in order to prevent a potential attacker from reading any communication content. This could be achieved with the addition of a symmetric cipher. There are also cipher modes (such as GCM = Galois Counter Mode) that combine authentication and

encryption. It is vital to mention that sole encryption cannot replace authentication, because encryption alone would still be susceptible to manipulation attacks.

Nonetheless, all the above-described methods require the presence of a shared, secret key on both the tester and the ECU. It is advisable that this secret key is different for each ECU and tester. This so-called

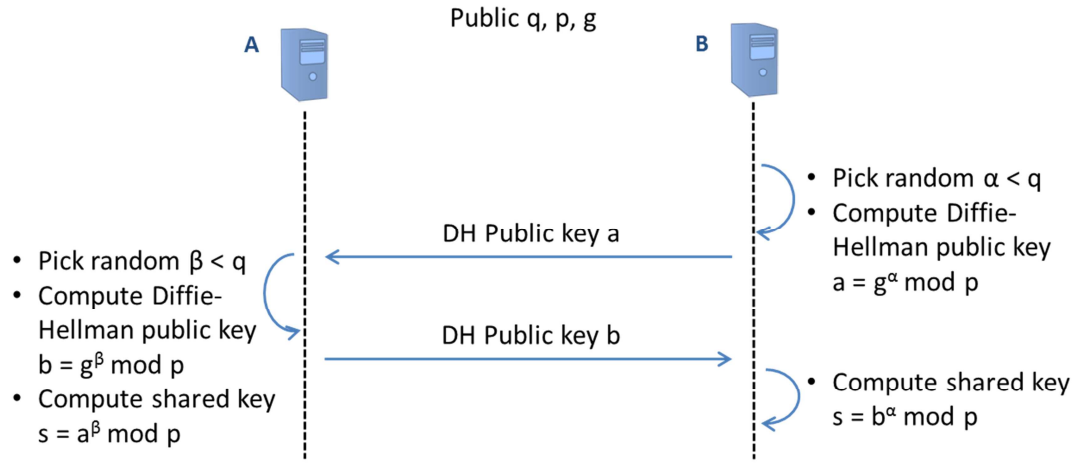

Figure 7: Classical Diffie-Hellman Key Exchange

"key diversification" prevents successful attacker to reuse the compromised secret key on other ECUs and testers (see also Section 3.4). Ideally, the secret key is also different for each session to prevent a single successful attack to be reusable for other sessions on the same ECU. To establish a secret key for each session between two parties a key agreement scheme can be used. One of the most widely used key agreement schemes is the Diffie-Hellman scheme.

Establishing a shared secret using the Diffie-Hellman key exchange scheme requires firstly a cyclic group over a finite field described by primes q, p and a generator g [10] Since these parameters are not private information, it is highly recommended to use some fixed, predefined parameters, which are considered not to have any known security weakness [11]. In the first step the ECU picks a random number α smaller than q and calculates the Diffie-Hellman public key $a = g^{\alpha} \bmod p$, and sends a to the tester. The tester picks β smaller than q and calculates the Diffie-Hellman public key $b = g^{\beta} \bmod p$, and sends b to the ECU. Both parties can then calculate the shared secret $s = b^{\alpha} = a^{\beta} \bmod p$ (cf. **Fehler! Verweisquelle konnte nicht gefunden werden.**).

Now both parties are in possession of shared secret s smaller than p and they can use a key derivation function to create a shared key for a symmetric encryption scheme. In the last step the ECU will send a key confirmation to the tester in order to verify that both sides created the same shared symmetric key. Note that this is necessary to thwart some classes of attacks, e.g. Man-in-the-Middle attack.

The whole *SecurityAccess* would then work as described in Section 4.1 up to the last "Seed-and-Key" protocol between the tester and the ECU. This step will follow the modified protocol, as described in **Fehler! Verweisquelle konnte nicht gefunden werden.**.

The Diffie-Hellman key exchange protocol was originally designed for cyclic groups over finite fields, but it is also possible to use an elliptic curves variant (ECDH). ECDH has the advantage that calculations are

much faster and the involved elements tend to be smaller for a similar level of security. The *SecurityAccess* works then as described above, only the calculations are done in elliptic curves.

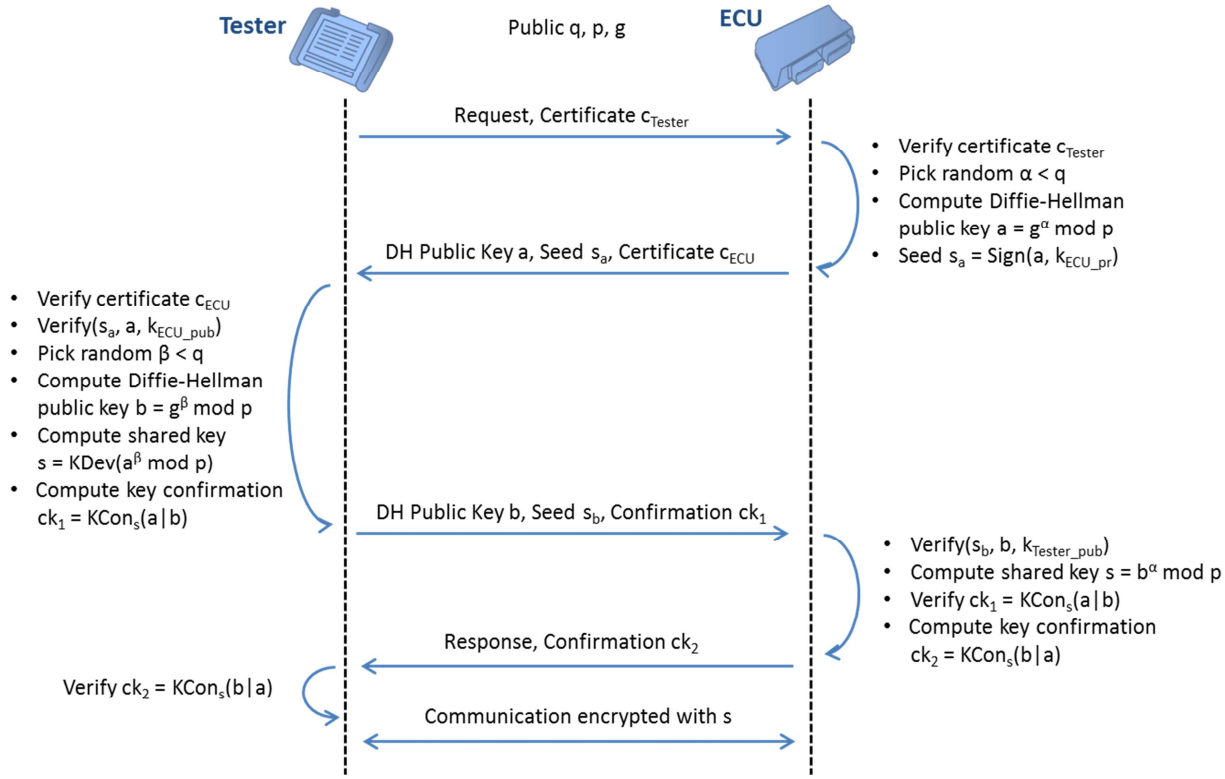

Figure 8: Authentication with Diffie-Hellman Key Exchange

4.3. OBD Firewall

A further mechanism to prevent hijacking of diagnostic sessions (cf. Section 3.3), is the usage of a firewall between the OBD interface and the in-vehicular network. A firewall detects and blocks malicious activity based on a rule set, e.g. a filter list, and therefore establishes a barrier between the untrusted OBD interface and the in-vehicular network. It essential, that the integrity and authenticity of the rule set is guaranteed, which can be achieved by applying a digital signature. This ensures that only the issuer of the rule set can change the data.

4.4. Secure OBD Log

The OBD logging functionality reports on all errors and malfunctioning behavior of a vehicle for further usage (cf. Section 2). Additionally, a log file should include all security-related events in order to detect possible attacks. In particular, at least all failed verifications attempts to an ECU shall be logged; however, it is recommended to log as well some successful and correct operations, for example, a successful tester authentication.

To prevent misuse of the log file, it is important to ensure the authenticity and the integrity as well as the confidentiality. An efficient and suitable security solution is the application of a symmetric cryptographic scheme based on a cipher mode that combines authentication and encryption, e.g. GCM (cf. Section 4.2).

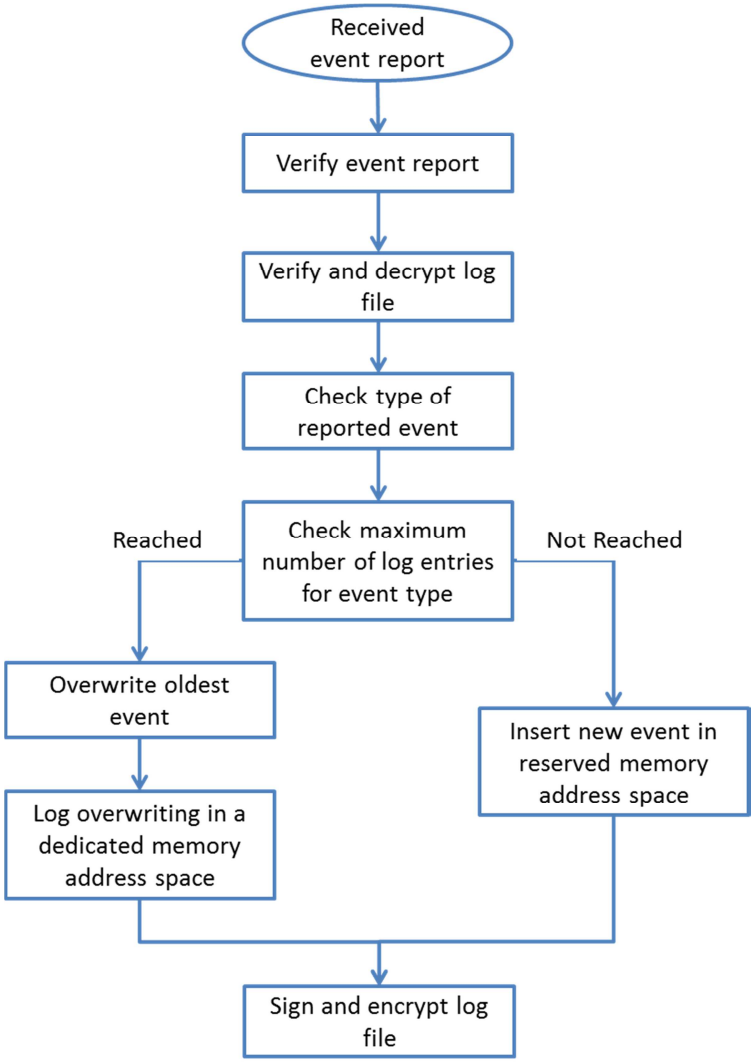

Figure 9: Flow-Chart of Secure OBD Log Entry.

The event logging application verifies the authentication tag and decrypts the log file at each access – either for reading or for insertion of a new entry. It is also possible to execute additional authentication verifications periodically and at start-up of the logging application.

Write access to the stored log file shall be strictly limited and only granted to the logging application to prevent any manipulations. Only the logging application shall be able to use the secret key that was used in the symmetric encryption and authentication scheme. Moreover, it is essential to ensure the security of the logging application itself. In practice, both access control to secret key and the establishment of a secure runtime environment can be realized with the help of a hardware security module as proposed in Section **Fehler! Verweisquelle konnte nicht gefunden werden.**.

However, even in case of a secured log file on the one hand and an authentic logging application on the other hand, an attacker or a malicious ECU might undermine the trustworthiness of the log file by flooding the logging mechanism with event reports. Due to the limited storage, a suitable concept for new entries in case of a full log file is required in order to minimize the risk of the erasure of important event

records. The proposed security concept can be combined with different mechanisms in order to protect the log file from malicious or faulty modification or erasure. First, events are categorized into dedicated types where each event type has its reserved memory address space within the log file. For example, if an event type refers to a dedicated application or service, this mechanism impedes a faulty application or service on an ECU to overwrite event records from another application or service. Second, the so-called “FIFO” principle is applied. FIFO refers to “First In First Out” and specifies a time-based strategy where the logging application overwrites the oldest event. The log file provides a dedicated memory address space especially for logging overwriting incidences. If the maximum number of log entries is reached and an old event is overwritten, then this is logged as event in this dedicated memory address space.

Additionally it is important to enable the logging application to prove that an event was actually reported by the claimed sender. To achieve this goal, the logging application needs to verify both the authenticity and integrity of the reported event. In case of an occurred event, first an authentication tag is computed over the event and then send to the logging application. The logging mechanism verifies the tag after receiving the reported event message. Only if the verification was successful, the logging process is started. The overall workflow to a secure OBD logging entry is depicted as an overview in Figure 9.

Next to the assurance of both an authentic log file and a secure logging application, only authorized user shall be able to access the file via the diagnostic interface. Chapter 4.1 introduced a secure user authentication and access control. The security concept is also suitable for access control to the OBD log file.

4.5. Smart Key Management

As discussed in Section 3.4, often the same secret key value is used globally for access to OBD functionality via “Seed-and-Key”. This allows an attacker to target many vehicles with no effort, since he has to extract the key only once. Thus, it is recommended to use individual keys for each ECU, since this would require complex and costly attacks for each ECU separately. However, if the “Seed-and-Key” algorithm is based on symmetric cryptography, this approach requires a very complex key distribution process. Since every tester needs to be able to access the secret key for each vehicle and each ECU individually as well. This issue can be solved by a “Seed-and-Key” algorithm, which is based on asymmetric cryptography. In this case, each tester and each ECU are in possession of their own private/public key pair, which addresses the recommendation of individual key.

Furthermore, since the there is no need to protect the confidentiality of the public part of the key, these values could be transferred between the entities for each *SecurityAccess* protocol. It is crucial to ensure that the public part of the key cannot be manipulated. This can be ensured by the use of certificates as proposed in the solutions to protect the *SecurityAccess* of diagnostic service (cf. Section 4.1), secure communication channel (cf. Section 4.2) and protected key storage (cf. Section 4.6).

The management of the certificates is not trivial, since certificates shall only be distributed to trusted parties and shall be able to be revoked in the case of misuse. It is recommended to assign this task to a professional key management solution, e.g. CycurKEYS (for more details, please refer to [12]).

4.6. Hardware-based Security

The previous chapter introduced cryptographic primitives in order to provide a secure diagnostic functionality. As mentioned in Section 3.5, the security of a cryptographic scheme is based on the secrecy of the key material. It is thus crucial to ensure the confidentiality of both stored and processed credentials by the provision of a protected and secure storage. Special hardware components or extensions are needed which provide confidential treatment of credentials and hardware-accelerated cryptographic functionality.

Common, standard ECUs possess only weak hardware protection against physical tampering. In addition, the introduced security mechanisms are based on asymmetric cryptographic operations. The common problem is that many of these cryptographic operations cannot be applied adequately in a general-purpose, performance-optimized ECU. This is especially an issue due to constraint in-vehicle restrictions regarding start-up time. Unfortunately, classical smartcard and trusted platform modules usually do not satisfy automotive requirements as, for instance physical resistance in presence of high temperatures or high costs caused by an additional external chip on the ECU. Efforts have been made to create suitable hardware security modules especially designed for in-vehicle integration, for example by the EVITA research project [13]. These automotive HSMs are integrated as System-on-a-Chip on ECUs in order to encapsulate security functions and provide necessary credentials. The HSM securely store keys and use them only internally to perform cryptographic operations – without ever outputting the keys themselves. Moreover, HSM provide additional features, which are essential for securing OBD.

- The integrated hardware-based random number generator can be used for generating random seed values within the “Seed and Key” protocol that is applied for secure user authentication and access control.
- Automotive HSM also support a secure boot process. Enforcing this feature by hardware, they act as a security anchor that starts a chain of trust in order to ensure the integrity and authenticity of further security mechanisms and to guarantee a trustworthy runtime environment for ECU applications.
- Some HSM counteract physical attacks as, for instance “Side-Channel Attacks” where an attacker aims to retrieve information about a secret key by measuring physical values during cryptographic operations.

Testers are easily accessible by a big group of people, for instance garage people; thus, there is an increased risk that the private key of the tester is extracted from the hardware by an attacker. Once in possession of this secret key the attacker can access diagnostic services without the tester. Hence, it is important to ensure a secure storage for keys.

Similar to ECUs normal tester possess only weak hardware protection. One possibility is to include a HSM in every tester; however, a HSM with no other security measures, e.g. password protected user accounts, allows only for one authentication key (cf. Section 3.6). Another possibility is the use of smartcard as key storage. Due to their widespread use, classical smartcards are among the best physically protected chips on the market. Well established and usually used for authentication tasks, they are suitable to store required credentials for diagnostic user authentication and access control. In particular, a smartcard securely stores and processes the tester’s private keys and the certificates. That means that the tester’s part of the applied “Seed-and-Key” authentication protocol – as proposed Section 4.1 – is mainly

done by the smartcard. Please note that from the ECU's point of view there is no difference if the tester uses a smartcard or not. Storing the credentials on smartcards has a further advantage. In case of multi-user sessions the key for each user can be saved on a separated smartcard and then used with one tester, instead of having multiple testers.

5. Conclusion & Outlook

This contribution gave a short overview of various security vulnerabilities, which arise from the unreflected usage of OBD and the UDS protocol. It was demonstrated that insufficient protection of the OBD interface leads to a security risk which is even today unacceptable and will be increased in the future. To counter the risk, different security solutions were proposed which can be used in compliance with OBD and UDS standard, and minimize the attack surface and therefore the risk of attacks.

However, there are still some open challenges, which need to be addressed in the near future. For example, Over-the-Air diagnosis enables for diagnostic session remotely either over WiFi or some other mobil communication. Since this increases the attack surface, an attacker could hijack a diagnostic session without physical access to the car, it is even more important to secure the remote diagnosis. Another open issue is the protection of over-the-air repair or over-the-air-software updates. The proposed solutions can be applied in this case as well, but special care is needed to handle the special challenges arising from the remote access, i.e. error prone communication channel, which can be easily eavesdropped and destroyed by an attacker.

There are also some legal issues to be considered which might be mandatory in near future [14], e.g. the requirement of an processes in order to provide a car owner the possibility to control the access to private data which is stored within the vehicle. This process needs to be established not only legally but also a secure, technical solutions is required.

6. Bibliography

[1] A. V. Thiemel, M. Janke and B. Steurich, "Tachomanipulation - Technisch Vorbeugen," *ATZelektonik,* vol. 02, 2013.

[2] K. Koscher, A. Czeskis, F. Roesner, S. Patel, T. Kohno, S. Checkoway, D. McCoy, B. Kantor, D. Anderson and H. a. o. Shacham, "Experimental security analysis of a modern automobile," *Security and Privacy (SP), 2010 IEEE Symposium on,* 2010.

[3] C. Miller and C. Valasek, "Adventures in Automotive Networks and Contol Units," illmatics.com/car_hacking.pdf, 2013.

[4] International Organization for Standardization (ISO), "ISO 14229-1: Road vehicles - Unified diagnostic services (UDS)," 2006.

[5] International Organization for Standardization (ISO), "ISO 11898: Road vehicles - Controller area network (CAN)," 2003.

[6] International Organization for Standardization (ISO), "ISO 14229-4: Unified diagnostic services on FlexRay implementation," 2012.

[7] G. Drenkhahn, "Fahrzeugdiagnose über Internet-Protokolle," 2008.

[8] S. Checkoway, D. MaCoy, B. Kantor, D. Anderson, H. Shacham, S. Savage, K. Koscher, A. Czeskis, F. Roesener and T. Kohno, "Comprehensive Experentimental Analyses of Automotive Attack Surfaces," 2011.

[9] S. Bayer, T. Enderle and P. Vyleta, "Demonstration of Secure On-Board Diagnostic," 2014.

[10] A. Menezes and P. v. O. S. Vanstone, Handbook of Applied Cryptography, 1996.

[11] A. Lenstra and E. Verheul, "Selecting cryptographic key sizes," *Journal of Cryptology,* 2001.

[12] ESCRYPT Embedded Security GmbH, "CycurKEYS," 27 July 2014. [Online]. Available: https://www.escrypt.com/products/cycurkeys/overview/?L=0.

[13] EVITA Project, "EVITA - E-safety vehicle intrusion protected applications," Fraunhofer SIT, 2013. [Online]. Available: http://www.evita-project.org/publications.html.

[14] carIT, "http://www.car-it.com/daten-machen-autos-zu-zeugen-der-anklage/id-0039111," *carIT,* 2014.